StudyFlow — Полное ТЗ (финальная версия)

Мультиагентный планировщик обучения | Версия 2.3 | Апрель 2026

---

1. Назначение системы

StudyFlow помогает людям, изучающим **несколько разнородных тем одновременно**, эффективно распределять время, **понимать новые темы** и не терять мотивацию.

Пример user flow:

```
Пользователь: "Python дедлайн завтра, история — неделя"
StudyFlow: план дня + простое объяснение темы + микрозадачи
```

Интерфейс: Streamlit Web App (localhost:8501) + Docker Compose.

---

2. Пользователи и Job Stories (приоритизированные)

Аудитория: студенты, самоучки, школьники (15–35 лет, 2+ темы параллельно).

#	Job Story	Приоритет
JS-1	Когда у меня несколько тем с дедлайнами, я хочу получить план дня с учётом сложности тем, моего ритма и перегрузки, чтобы не провалить ничего важного.	**Must**
JS-2	Когда я не понимаю новую тему, я хочу получить простое объяснение с примерами и постепенным переходом от базового к сложному, чтобы быстро в ней разобраться.	**Must**
JS-4	Когда я застрял на сложной теме, я хочу 10-минутную микрозадачу по другой теме, чтобы вернуться с свежей головой и не бросить учёбу.	**Should**
JS-6	Когда я пропустил день учёбы, я хочу скорректированный план без чувства вины, чтобы вернуться в ритм за 1 сессию.	**Should**
JS-3	Когда я учу похожие концепции в разных предметах, я хочу объяснение через аналогии между ними, чтобы лучше запомнить обе темы сразу.	**Could**
JS-5	Когда я закончил блок материала, я хочу знать следующий шаг именно для моего уровня, чтобы не блуждать в контенте часами.	**Could**

---

3. Архитектура (3 агента + LangGraph)

```
┌──────────────┐
│ Streamlit UI │ ← localhost:8501
│   (Docker)   │
└──────┬───────┘
       │ REST API
       ▼
```

```
┌──────────────────┐
│   Supervisor     │ qwen2.5:7b
│   (Router)       │
└──────────────────┘
          │
          │
   ┌──────┴──────┐
   ▼             ▼
┌─────────┐   ┌──────────┐
│ Planner │   │   Tutor  │
│ Agent   │   │  Agent   │
│(3b)     │   │(kimi K2.5)│
│         │   │(Cloud)   │
└─────────┘   └──────────┘
      │             │
      └─────┬───────┘
            ▼
      ┌──────────────┐
      │  Evaluator   │ qwen2.5:1.5b
      │ (Quality)    │
      └──────────────┘
            │
            ▼
┌──────────────┐
│ ChromaDB +   │
│ Langfuse     │
└──────────────┘
```

LangGraph flow:

```
User → Supervisor → [Planner|Tutor] → Evaluator → User
Evaluator score < 0.8 → retry Worker
```

---

4. Технологический стек

Компонент	Инструмент	Docker сервис
LLM Engine	Ollama	`ollama`
Модели	qwen2.5:7b (supervisor), qwen2.5:3b , kimi-k2.5, qwen2.5:1.5b (eval)	`./models/`
Оркестрация	LangGraph + LangChain	`backend`
Vector DB	ChromaDB	`chromadb`
UI	Streamlit	`streamlit`
Observability	Langfuse	`langfuse`
Backend API	FastAPI	`backend`
Изоляция	Docker Compose	все сервисы

---

5. Скиллы (Tools)

Planner Agent (JS-1, JS-4, JS-6)

- `search_topics()` — список тем и дедлайнов
- `analyze_complexity(topic)` — оценка сложности материала
- `build_timeline(topics, time_window)` — план дня/недели
- `microtask(topic, duration=10)` — короткая задача
- `adjust_plan_after_miss(topics)` — корректировка после пропуска

Tutor Agent (JS-2, JS-3)

- `explain_topic(topic, level)` — простое объяснение темы
- `rag_search(query)` — поиск в материалах пользователя
- `extract_concepts(text)` — ключевые идеи из текста
- `find_analogies(topic1, topic2)` — связи между темами

Evaluator Agent (все stories)

- `verify_facts(response, sources)` — проверка галлюцинаций
- `assess_relevance(query, response)` — полезность ответа
- `quality_score()` — финальная оценка (0-1)

---

6. Системные промпты

Supervisor.md:

```
Ты — оркестратор StudyFlow. Классифицируй запрос:

1. ПЛАНИРОВАНИЕ/дедлайны/микрозадачи → Planner Agent
2. ОБЪЯСНЕНИЕ темы/аналогии → Tutor Agent
3. ОБОИ → оба параллельно

ВСЕГДА передавай Evaluator перед финальным ответом.
Формат: {"agent": "planner", "params": {...}}
```

Planner.md:

```
Ты помогаешь людям эффективно учиться по нескольким темам.
Создавай реалистичные планы с учётом дедлайнов и энергии.
Формат ответа всегда JSON.
```

Tutor.md:

```
Ты объясняешь сложные темы простым языком с примерами.
Начинай с базового уровня, постепенно усложняй.
Ищи связи между темами для лучшего запоминания.
```

Evaluator.md:

```
КРИТИЧЕСКАЯ проверка:
```

```
✅ Факты из источников (не галлюцинации): да/нет
✅ Решает проблему пользователя: да/нет
✅ Конкретен и полезен: да/нет

Score < 0.8 = RETRY
```

---

7. Docker Compose

```yaml
version: '3.8'
services:
  ollama:
    image: ollama/ollama
    ports: ["11434:11434"]
    volumes: ['./models:/root/.ollama']

  chromadb:
    image: chromadb/chroma
    ports: ["8001:8001"]
    volumes: ['./chroma:/chroma/chroma']

  langfuse:
    image: ghcr.io/langfuse/langfuse
    ports: ["3000:3000"]
    environment:
      - CLICKHOUSE_PASSWORD=studyflow
      - NEXTAUTH_SECRET=studyflow123

  backend:
    build: ./backend
    ports: ["8000:8000"]
    depends_on: [ollama, chromadb, langfuse]
    volumes: ['./backend:/app']

  streamlit:
    build: ./ui
    ports: ["8501:8501"]
    depends_on: [backend]
    volumes: ['./ui:/app']
```

---

8. Память

Тип	Реализация	Что хранит
Краткосрочная	ConversationSummaryBufferMemory	Диалог сессии
Долгосрочная	ChromaDB	Конспекты, PDF по темам
Профиль	JSON + EntityMemory	Темы, дедлайны, продуктивность

---

9. Evals

Метрика	Цель	Метод
Plan Accuracy	>85%	Выполнение плана
Explanation Quality	>4/5	Понятность объяснений
Evaluator Reject Rate	10–20%	% retry
P95 Latency	<25s	Langfuse

---

10. Бэклог (MoSCoW)

Эпик 1: Core MVP (Must)

```
S1.1: /plan endpoint + Planner Agent [AC: JSON план]
S1.2: /explain endpoint + Tutor Agent [AC: простое объяснение]
S1.3: Streamlit UI план+чат [AC: 2 рабочих сценария]
```

Эпик 2: Quality (Should)

```
S2.1: Evaluator Agent [AC: блокирует 15% ответов]
S2.2: PDF загрузка + RAG [AC: поиск по конспектам]
S2.3: Микрозадачи [AC: 10мин задачи]
```

Эпик 3: Advanced (Could)

```
S3.1: Аналогии между темами
S3.2: Корректировка плана
```

---

11. Структура репо

```
studyflow/
├── backend/
│   ├── agents/     # supervisor.py, planner.py, tutor.py, evaluator.py
│   ├── skills/     # tools для каждого агента
│   ├── graph.py    # LangGraph workflow
│   └── main.py     # FastAPI
├── ui/             # Streamlit app.py
├── prompts/        # *.md файлы
├── models/         # Ollama
├── chroma/         # Vector DB
└── docker-compose.yml
```

---

12. Демо-скрипт защиты

```
1. docker-compose up (20сек)
2. localhost:8501 → "Python завтра, SQL неделя" → план дня
```

```
3. localhost:8501 → "Объясни JOIN простыми словами" → объяснение
4. localhost:3000 → Langfuse traces (Supervisor→Planner→Evaluator)
5. docker ps → архитектура
```